



Practical 1: The Study Squad Tracker

COS 132 — Imperative Programming

Practical Specification

Programme note. The premium COS 132 course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. This material is Day 1 of the 7-day programme.

Module: COS 132 – Imperative Programming **Set by:** Swift Tuition (ST2 Programme, Day 1) **Release Date:** 27 April 2026 **Due Date:** 3 May 2026 at 23:59 (end of Day 7) **Total Marks:** 97

Read the entire specification before starting with the practical.

Contents

1. Overview
 2. Your Task
 - 2.1 Squad
 - 2.1.1 Variables
 - 2.1.2 Functions
 3. Implementation Details
 4. How to Use the Codespace
 5. Submission
-

1 Overview

Every weekday evening, three students meet in the Engineering Library to work through COS 132 with their tutor: **Caleb**, **Yolisa** and **Sahana**, plus **Swift**, who has been their tutor since the start of the year. Caleb is also an actual uncle himself, his sister's kid started walking last week. None of that stops him calling Swift "Uncle" every five minutes. Swift has stopped correcting him.

Yolisa has decided that the squad needs a proper attendance and quiz tracker. Not just any tracker, but one whose summary report prints with exactly the right spacing, because Yolisa is Yolisa about that kind of thing. Sahana wants the system to flag who has been showing up consistently. Caleb wants to know the exact moment he qualifies to tutor Yolisa back, just so he can hold it over her head.

Sexy Swift sensibly says: "Let's just write a program."

You are going to write that program.

You will build a small library called Squad that lives inside a namespace called SQUAD. The library tracks four pieces of information per member (name, total study hours, sessions attended, quiz average) and exposes functions that work with those values. A test harness in `main.cpp` is provided. You only edit `Squad.cpp`.

A starter `Squad.cpp` is provided with stub implementations for every function. You complete the bodies. The header file `Squad.h` and the test harness `main.cpp` are also provided and must not be changed.

The four extern variables are already defined in the starter `Squad.cpp` with their initial values, so you do not need to add them yourself.

1.1 Mark Breakdown

The following table shows the marks assigned to each group of functions on the Swift Tuition grader:

Task	Marks
Arithmetic Functions	26
Boolean Functions	30
Overloaded Function	19
Report Function	10
Orchestrator Function	12
Total	97

2 Your Task

You must provide the implementations for every function declared in `Squad.h`, inside `Squad.cpp`. All implementations must strictly follow the specifications in the sections below.

For the notation used in this section, please note that the `_` indicates a single space, `$` indicates a newline, and a variable between `{` and `}` is the variable you need to print out.

Ensure that your namespace is called `SQUAD` to avoid confusion.

2.1 Squad

This namespace contains the variables and functions that the squad uses to track their study sessions.

2.1.1 Variables

2.1.1.1 `extern std::string memberName`

- This variable represents the name of the squad member currently being processed.
- Initialise this variable to be an empty string `""`.

2.1.1.2 `extern int totalHours`

- This variable represents the total number of hours a member has studied.
- Initialise this variable to be `0`.

2.1.1.3 `extern int sessionsAttended`

- This variable represents the number of sessions a member has attended.
- Initialise this variable to be `0`.

2.1.1.4 `extern double quizAverage`

- This variable represents the member's quiz average as a percentage.
- Initialise this variable to be `0.0`.

Note: the extern declarations live in `Squad.h` and the matching definitions (with their initial values) are already in the starter `Squad.cpp`. You do not need to add or change them. They are listed here for reference, because some marks may depend on you using these existing variables in later functions.

2.1.2 Functions

2.1.2.1 `int addHours(int currentTotal, int newHours)`

- This function adds the hours from a single session to a running total.
- Returns the sum of `currentTotal` and `newHours`.

2.1.2.2 int incrementSessions(int currentCount)

- This function increases the number of sessions attended by one.
- Returns `currentCount + 1`.

2.1.2.3 double calculateAverage(double total, int count)

- This function calculates the mean of a running total over a number of items.
- If `count` is 0, returns 0.0 to avoid division by zero.
- Otherwise, returns `total / count`. The division must be done as a floating-point division, not an integer division.

2.1.2.4 double scaleHours(int hours, double factor)

- This function scales a number of hours by a factor (used during exam week, when study time is multiplied).
- Returns the product of `hours` and `factor`. Note that `hours` is an `int` and `factor` is a `double`. The result must be a `double`.

2.1.2.5 bool isOnFire(int sessionsAttended)

- This function checks whether a member is on a streak.
- Returns `true` if `sessionsAttended` is greater than or equal to 5. Otherwise returns `false`.

2.1.2.6 bool isPassing(double quizAverage)

- This function checks whether a member is passing.
- Returns `true` if `quizAverage` is greater than or equal to 50.0. Otherwise returns `false`.

2.1.2.7 bool canTutor(double quizAverage, int sessionsAttended)

- This function checks whether a member is allowed to tutor someone else. Caleb has been asking about this function repeatedly.
- Returns `true` only if BOTH of the following are true:
 - `quizAverage` is greater than or equal to 75.0
 - `sessionsAttended` is greater than or equal to 8
- Otherwise returns `false`.

2.1.2.8 void setReminder(int hours)

- This function prints a study reminder, assuming the input is already in hours.
- Output format: `Reminder_set_for_{hours}h$`
- For example, if `hours` is 3, the output is `Reminder set for 3h` followed by a newline.

2.1.2.9 void setReminder(int value, std::string unit)

- This is an overloaded version of setReminder that accepts a unit.
- If unit is "hours", treat value as already being in hours.
- If unit is "minutes", convert by integer-dividing value by 60 (so 120 minutes becomes 2 hours, and 90 minutes becomes 1 hour).
- The grader will only ever pass "hours" or "minutes" as the unit. You do not need to handle other strings.
- After the conversion (or no conversion), output uses the same format as the single-argument version: Reminder_set_for_{hours}h\$.

2.1.2.10 void printSessionLog(std::string name, int sessions, int hours, double average)

- This function prints a member's session log in a strict format.
- Output format (exactly as shown, including the dashes):

```
-----SQUAD_MEMBER_LOG-----$
Name:_{name}$
Sessions:_{sessions}$
Hours:_{hours}$
Average:_{average}$
-----$
```

- For example, if name is Yolisa, sessions is 10, hours is 24, and average is 80.0, the output is:

```
-----SQUAD_MEMBER_LOG-----
Name: Yolisa
Sessions: 10
Hours: 24
Average: 80
-----
```

- Use default cout formatting for the average value. Do not add <iomanip> manipulators (std::fixed, std::setprecision, etc.) — these are not in the allowed headers anyway. With default formatting, 80.0 prints as 80 and 78.5 prints as 78.5. The grader expects this exact behaviour.

2.1.2.11 bool processMember(std::string name, int sessions, int hours, double average)

- This function orchestrates the per-member status report. It runs the existing functions in the right order, prints the results, and returns whether the member is a "Squad Star".
- The function must do exactly the following, in this order:
 1. Call printSessionLog(name, sessions, hours, average) to print the log header.
 2. Compute three booleans by calling the existing functions: isOnFire(sessions), isPassing(average), canTutor(average, sessions).
 3. Print each result on its own line, in this exact order:
 - On_fire:_{result of isOnFire}\$
 - Passing:_{result of isPassing}\$
 - Can_tutor:_{result of canTutor}\$

4. Return true if all three booleans are true. Otherwise return false.
- Boolean values print as 1 for true and 0 for false when sent to cout.
-

3 Implementation Details

- You must implement the functions in the namespace exactly as stipulated in this specification. Failure to do so will fail the corresponding tests.
 - You may only use **C++98**.
 - You may only use the following libraries:
 - `<iostream>`
 - `<string>`
 - Do **not** include using `namespace std;` in any of the files.
 - Do **not** modify `Squad.h` or `main.cpp`. The grader uses the original versions and will overwrite any changes you make to these.
-

4 How to Use the Codespace

When you open this practical in GitHub Codespaces (or clone it locally), the layout is:

```
p1-study-squad-tracker/starter/  
Squad.h          <- header, do not change  
Squad.cpp        <- you implement here  
main.cpp         <- test harness, do not change  
Makefile         <- build configuration  
grade.sh         <- the grading script  
tests/  
  inputs/        <- input files for each test case  
  expected/      <- expected output for each test case  
README.md        <- instructions
```

The day-to-day workflow is:

1. Open `Squad.cpp` and implement the functions.
2. From the terminal, run `./grade.sh`. The script compiles your code and runs every test, printing a pass/fail report with marks.
3. Read the failures, edit `Squad.cpp`, run `./grade.sh` again. Repeat until everything passes.

You can also compile manually with `make` and run the program with `./squad` if you want to test specific cases by hand. The `README.md` in the starter folder explains both options in detail.

5 Submission

This is a Swift Tuition practical, not an official UP submission. There is no FitchFork upload and no zip archive.

Your final mark for this practical is the score reported by the most recent run of `./grade.sh` against your `Squad.cpp`. You can run it as many times as you want, with no penalty. The practical stays available for the entire seven days of the ST2 programme, with no late deadline.

When you are happy with your score, your work is done. There is nothing to submit.